An Approach to the Discovery of Accurate and Expressive Fix-Time Prediction Models

Francesco Folino(✉), Massimo Guarascio, and Luigi Pontieri

ICAR-CNR, National Research Council of Italy,
via P. Bucci 41C, 87036 Rende, CS, Italy
{ffolino,guarascio,pontieri}@icar.cnr.it

Abstract. Predicting the fix time (i.e. the time needed to eventually solve a case) is a key task in an issue tracking system, which attracted the attention of data-mining researchers in recent years. Traditional approaches only try to forecast the overall fix time of a case when it is reported, without updating this preliminary estimate as long as the case evolves. Clearly, the actions performed on a case can help refine the prediction of its (remaining) fix time, by using Process Mining techniques, but typical issue tracking systems lack task-oriented descriptions of the resolution process, and store fine-grain records, just registering case attributes' updates. Moreover, no general approach has been proposed in the literature that fully supports the definition of high-quality derived data, which were yet proven capable to improve prediction accuracy considerably. A new fix-time prediction framework is presented here, along with an associated system, both based on the combination of two kinds of capabilities: *(i)* a series of modular and flexible data-transformation mechanisms, for producing an enhanced process-oriented log view, and *(ii)* several induction techniques, for extracting a prediction model from such a view. Preliminary results, performed on the logs of two real issue tracking scenarios, confirm the validity and practical usefulness of our proposal.

Keywords: Data mining · Prediction · Business process analysis · Bug tracking

1 Introduction

Issue tracking systems (a.k.a. "trouble/incident ticket" systems) are widely used in real collaboration environments, in order to manage, maintain and help resolve various issues arising within an organization or a community.

An important problem in such a context concerns the prediction of the fix time, i.e. the time needed to solve a case. Several data-mining works [1,9,10] recently approached this problem in the specific field of bug tracking systems, a popular sub-class of issue tracking systems, devoted to support the notification and fixing of bugs affecting some software artifact, within a software development/maintenance community.

© Springer International Publishing Switzerland 2015
J. Cordeiro et al. (Eds.): ICEIS 2014, LNBIP 227, pp. 108–128, 2015.
DOI: 10.1007/978-3-319-22348-3_7

The approaches proposed so far mainly try to induce discrete (i.e. classification-oriented) or continuous (i.e. regression-oriented) predictors from historical bug logs, by resorting to classical propositional prediction methods, and simply regards each bug case as a tuple, encoding all information available when the bug was initially reported, and labelled with a discrete or numerical (target) fix-time value. Therefore, all the data recorded along the life of a bug/issue case (e.g., changes made to properties like its priority, criticality, status, or assignee) are completely disregarded, despite they could turn useful for refining, at run-time, the prediction of (remaining) fix times.

The discovery of predictive (state-aware) process models [4,5,13] is a young line of research in the field of Process Mining. Unfortunately, the approaches developed in that field assume that log records be mapped to well-specified process tasks, which are, instead, hardly defined in real bug/issue systems, where the logs only store the sequence of changes made to a case's attributes. In fact, despite many systems support the design of workflows, these are rarely used in real applications. Moreover, different repositories tend to exhibit heterogeneous data schemes, even if developed over the same platform (e.g., Bugzilla).

In this work, we right attempt to overcome the limitations of current solutions by proposing a novel approach to the discovery of (state-aware) fix-time prediction models, out of low-level bug/issue logs, which can fully exploit all information stored in the form of both case attributes and attribute modification records. In order to reach a suitable abstraction level over case histories, a modular set of parametric data-transformation methods is defined, which allow to convert each case into a process trace (with update records abstracted into high-level activities), and to possibly enrich these traces with derived/aggregated data. In particular, the analyst is offered the possibility to effectively exploit textual descriptions, by summarizing them into a lower-dimensional concept space, with the help of SVD-based techniques. A high-quality process-oriented view of log data is so obtained, which can be analyzed with predictive process mining methods, to eventually induce a predictive process model, capable of supporting fix-time forecasts, at run-time, for future cases. The approach has been implemented into a system, provided with an integrated and extensible set of data-transformation and predictive learning tools, along with a series of functionalities for browsing, evaluating and analysing the discovered process models.

The remainder of the paper is organized as follows. After discussing relevant related works in Sect. 2, we introduce some basic concepts and notation in Sect. 3. Several core event manipulation methods and trace manipulation ones are presented in Sects. 4 and 5, respectively. Section 6 illustrates the whole approach to the discovery of predictive fix-time models, while Sect. 7 its current implementation. We finally discuss a series of tests in Sect. 8, and draw a few concluding remarks in Sect. 9.

2 Related Work

As mentioned above, previous fix-time prediction works focus on the case of bug tracking systems, and resort to classical learning methods, conceived for analyzing propositional data labelled with a discrete or numerical target.

Random-forest classifiers and linear regressors were trained in [9] and in [1], respectively, in order to predict bug lifetimes, using different bug attributes as input variables. Different standard classification algorithms were employed instead in [10] to the same purpose. Decision trees were also exploited in [7], in order to estimate how promptly a new bug report will receive attention. Standard linear regression was also used in [8] to predict whether a bug report will be triaged within a given amount of time.

As mentioned above, none of these approaches explored the possibly of improving such a preliminary estimate subsequently, as long as a case undergoes different treatments and modifications. The only (partial) exception is the work in [10], where some information gathered after the creation of a bug is used as well, but just for the special case of unconfirmed bugs, and till they were definitely accepted. Conversely, we want to exploit the rich amount of log data stored for bug/issue cases (across their entire life), in order to build a history-aware prediction model, capable of providing accurate run-time forecasts for the remaining fix time of new (unfinished) cases.

Predicting processing times is the goal of an emerging research stream in the field of Process Mining, which specifically addresses the induction of state-aware performance model out of historical log traces. In particular, the discovery of an annotated finite-state model (AFSM) was proposed in [13], where the states correspond to abstract representations of log traces, and store processing-time estimates. This learning approach was combined in [4,5] with a predictive clustering scheme, where the initial data values of each log trace are used as descriptive features for the clustering, and its associated processing times as target features. By reusing existing induction methods, each discovered cluster is then equipped with a distinct prediction model – precisely, an AFSM in [4], and classical regression models in [5].

Unfortunately, these Process Mining techniques rely on a process-oriented representation of system logs, where each event refers to a well-specified task; conversely, common bug tracking systems just register bug attribute updates, with no link to resolution tasks. In order to overcome this limitation, we want to help the analyst extract high-level activities out of bug/issue histories, by providing her/him with a set of data transformation methods, tailored to fine-grain attribute-update records, like those stored in typical bug/issue tracking systems.

The effectiveness of derived data in improving fix-time predictions was pointed out in [2], where a few summary statistics and derived properties were computed for certain Bugzilla repositories, in a pre-processing phase. We attempt to generalize such an approach, by devising an extensible set of data transformation and data aggregation/abstraction mechanisms, allowing to extract and evaluate such derived features from a generic bug/issue resolution trace.

3 Basic Concepts and Notation

We next illustrate the structure and main characteristics of issue/bug resolution logs, which are the source for extracting a process-oriented fix-time prediction model.

For the sake of concreteness, let us focus on the structure of bug repositories developed with *Bugzilla* (http://www.bugzilla.org), a general-purpose bug-tracking platform, devoted to support people in various bug-related tasks – e.g., keep track of bugs, communicate with colleagues, submit/review patches, and manage quality assurance (QA). We pinpoint that this choice does not limit the generality of our approach, since similar strategies take place in most real bug/issue -tracking applications.

Usually bug/issue resolution tasks are carried out in an unstructured fashion, without being enforced by a prescriptive process model. Such applications mainly looks like a repository, where an extensible set of attributes are kept for a case (concerning the resolution of a bug/issue), and possibly updated along the entire life of that case.

For example, in the Bugzilla repository of project *Eclipse* (also used in our tests), the main attributes associated with each bug b are: the `status` and `resolution` of b; who entered b into the system (`reporter`); the last solver b was assigned to (`assignee`); the `component` and `product` affected by b; b's `severity`'s and `priority`'s levels; a `milestone`; the list of users to be kept informed on b's progress (`CC`); the lists of other bugs depending on b (`dependsOn`), and of related documents (`seeAlso`).

Some attributes (e.g., `reporter`) are static, whereas others (e.g., `assignee`, `status`, `resolution`) may change while a case evolves. In particular, the `status` of a bug b may take the following values: *unconfirmed* (i.e. b was reported by an external user, and it must be confirmed by a project member), *new* (i.e. b was opened/confirmed by a project member), *assigned*, *resolved* (i.e. a fix was made to b, but it needs to be validated), *verified* (i.e. a QA manager has validated the fix), *reopened* (if the last fix was insufficient), and *closed*. For a resolved bug b, the `resolution` field may take one of these values: *fixed*, *duplicate* (i.e. b is a duplicate of another bug), *works-for-me* (i.e. b has been judged unfounded), *invalid*, *won't-fix*.

In any Bugzilla repository, the whole history of a bug is stored as a list of update records, all sharing the same structure, which consists five predefined fields (in addition to a bug identifier): *who* (the person who made the update), *when* (a timestamp for the record), *what* (the attribute modified), *removed* (the former value of that attribute) and *added* (the newly assigned value).

Figure 1 reports, as an example, all the update records of a single Ecplise's bug.

Traces and Associated Data. The contents of a bug/issue tracking repository can be viewed as a set of *traces*, each storing the sequence of *events* recorded during a case. As explained above, each of these events concerns a modification to a case attribute, and takes the form of the records in Fig. 1.

Who	When	What	Removed	Added
svihovec	2012-06-20 10:12:27 EDT	CC		margolis, svihovec
		Target Milestone	---	0.8.1 Final
jinfahua	2012-06-20 22:21:22 EDT	CC		jinfahua, pfyu
		Assignee	edt.ide.ui-inbox	songfan
pfyu	2012-06-21 03:52:44 EDT	Attachment #217671 Flags		review?
svihovec	2012-06-21 11:09:34 EDT	CC		jspadea, jvincens
pfyu	2012-06-25 05:04:17 EDT	CC		wxwu
jspadea	2012-07-02 10:26:15 EDT	Assignee	songfan	jspadea
jspadea	2012-07-02 15:21:21 EDT	Status	NEW	RESOLVED
		Resolution	---	FIXED
lasher	2012-07-18 15:27:24 EDT	Attachment #218182 Flags		iplog+
mheitz	2013-01-03 10:44:55 EST	Status	RESOLVED	CLOSED

Fig. 1. Activity log for a single Bugzilla's bug (whose ID is omitted for brevity). Row groups gather "simultaneous" update records (sharing the same timestamp and executor).

Let E be the universe of all possible events, and $\mathcal{T}$ be the universe of all possible traces defined over them. For any event $e \in E$, let $who(e)$, $when(e)$, $what(e)$, $removed(e)$, and $added(e)$ be the executor, the timestamp, the attribute modified, the former value and new value stored in e, respectively. In the case of Bugzilla repositories, each of these dimensions just corresponds to a label. This does not happen, however, in other issue tracking systems, where an event record can encode a change to multiple attributes, and gather different kinds of information on the organizational resource involved (e.g., her/his role, team, department, etc.). In such a case, the representation of log events can be generalized, by letting $who(e)$, $when(e)$, $what(e)$, $removed(e)$, and $added(e)$ take the form of a tuple (over some suitable attribute space).

For each trace $\tau \in \mathcal{T}$, let $len(\tau)$ be the number of events in τ, $\tau[i]$ be the i-th event of τ, for any $i = 1 \,..\, len(\tau)$, and $\tau(i] \in \mathcal{T}$ be the *prefix* trace gathering the first i events in τ.

Prefix traces have the same form as fully unfolded ones, but only represent partial case histories. Indeed, the prefix traces of a case depicts the evolution of the case across its whole life. For example, the activity log of Fig. 1 (which just stores the history of one bug) will be represented as a trace τ_{ex} consisting of 12 events, one for each of the update records (i.e. rows of the table) in the figure; in particular, for the first event, it is $who(\tau_{ex}[1]) = svihovec$, $what(\tau_{ex}[1]) = \texttt{CC}$, $added(\tau_{ex}[1])) = \{margolis, svihovec\}$.

As mentioned above, typical tracking systems store several attributes for each case (e.g., `reporter`, `priority`, etc.), which may take different values during its life. Let $F_1, \ldots, F_n$ be all of the attributes defined for a bug. Then, for any (either partial of completed) trace τ, let $data(\tau)$ be a tuple storing the updated values of these attributes associated with τ (i.e. the values taken by the corresponding case after the last event of τ), and $data(\tau)[F_i]$ be the value taken by F_i (for $i = 1..n$).

Finally, a *log* L is a finite subset of $\mathcal{T}$, while the *prefix set* of L, denoted by $\mathcal{P}(L)$, is the set of all possible prefix traces that can be extracted from L.

Fix-time Measurements and Models. Let $\hat{\mu}_F : \mathcal{T} \to \mathbb{R}$ be an unknown function assigning a fix-time value to any (sub-)trace. The value of $\hat{\mu}_F$ is known over all $\mathcal{P}(L)$'s traces, for any given log L; indeed, for any trace τ and prefix $\tau(i]$, it is $\hat{\mu}_F(\tau(i]) = when(\tau[len(\tau)]) - when(\tau[i])$. For example, for the trace $\tau_{ex}(2]$ encoding the first 2 events in Fig. 1, it is $\hat{\mu}_F(\tau_{ex}(2]) = when(\tau[12]) - when(\tau[2]) =$ 197 days.

A *Fix-time Prediction Model (FTPM)* is a model for estimating the remaining fix time of a case, based on its current trace. Learning such a model, which approximates the unknown function $\hat{\mu_F}$, is an induction task, where log L plays as the training set, and the value $\hat{\mu}_F(\tau)$ of the target measure is known for each (prefix) trace $\tau \in \mathcal{P}(L)$.

4 Event Manipulation Operators

In the discovery of an $FTPM$ model we want to consider both case histories (i.e. all sequences of update records) and case properties (e.g., the affected product, severity level, reporter). To this end, we will regard some of the actions performed on a case as a clue for the activities of an unknown (bug/issue resolution) process, in order to eventually exploit Process Mining techniques. In fact, simply defining such activities as all possible changes to the status of a case would lead to discarding relevant events, such as the (re-)assignment of the case to a solver, or the update of key properties (like its severity, criticality, or category). On the other hand, we do not either want to look at all attribute updates as resolution tasks, since many of them are hardly linked to fix times, and they may even produce a noise-like effect on the discovery of fix-time predictors.

4.1 Activity-Oriented Event Abstraction

An event abstraction function α is a function mapping each event $e \in E$ to an abstract representation $\alpha(e)$, which captures relevant facets of the action performed. In current process mining approaches, log events are usually abstracted into their associated tasks, possibly combined with other properties of them (e.g., their executors), under the assumption that the events correspond to the execution of work-items, according to a workflow-oriented view of the process analyzed.

In our framework, such a function α is intended to turn each issue/bug -tracking event into a high-level bug/issue-resolution activity, by mapping the former to a label that captures well its meaning. Since only attribute-update events are tracked, the analyst is allowed to define this function in terms of their fields (i.e. *who*, *when*, *what*, *added*, and *removed*).

The default instantiation of α, denoted by $\overline{\alpha}$ and conceived for Bugzilla's records, is defined as follows (with + denoting string concatenation):

$$\overline{\alpha}(e) = \begin{cases} what(e) + \text{":="} + added(e), \text{ if } what(e) \in \{\texttt{status}, \texttt{resolution}\} \\ \text{"}\Delta\text{"} + what(e), \text{ otherwise} \end{cases} \quad (1)$$

This particular definition of α focuses on what attribute was modified, while abstracting any other event's field (namely, *who*, *when*, *removed*, and *added*); as an exception, the assigned value is included in the abstract representation when the update concerns the `status` or `resolution`, as such information, characterizing the state of a bug, can improve fix-time predictions. For example, for the first two events of trace τ_{ex} (gathering all the records in Fig. 1), it is $\overline{\alpha}(\tau_{ex}[1]) = \Delta$`CC`, and $\overline{\alpha}(\tau_{ex}[2]) = \Delta$`TargetMilestone`, while the activity label of the last event ($\tau_{ex}[12]$) is `status`:=*closed*.

Different event abstraction functions can be defined by the analyst, in order to focus on other facets of bug activities, or to change the level of detail, depending on the specific bug attributes (and associated domains) available in the application scenario at hand. For instance, with regard to the scenario of Sect. 3, one may refine the representation of severity-level changes by defining two distinct activity labels for them, say Δ*Severity-Eclipse* and Δ*Severity-NotEclipse*, based on the presence of substring "eclipse" in the e-mail address of the person who made the change.

4.2 Macro-Events' Restructuring

In real bug tracking environments, multiple fields of a case are often modified in a single access session, and the corresponding activity records are all stored with the same timestamp, in a rather arbitrary order. For example, in our tests, we encountered many cases where the closure of the bug (i.e. an event of type `status`:=*closed*) preceded a "contemporaneous" change to the assignee (or a message dispatch).

Regarding each set of contemporaneous events as one *macro-event*, the analyst can define three kinds of data-manipulation rules, in order to rearrange them based on their fields: *(i)* a *predominance* rule, assigning different relevance levels to simultaneous events (with the ultimate aim of purging off less relevant ones); *(ii)* a set of *merging* rules, indicating when two or more contemporaneous events (with the same predominance level) must be merged together, and which activity label must be assigned to the resulting aggregated event; *(iii)* a set of *sort* rules, specifying an ordering relation over (non-purged and non-merged) simultaneous events.

Any combination of the above kinds of rules will be collectively regarded, hereinafter, as a *macro-event criterion*. The default instantiation of this criterion, tailored to Bugzilla's bugs, is summarized in Table 1, where each event is given a "predominance" level, only based on *what* attribute was updated in the event. Such levels acts as a sort of priority score in the selection of events (the lower the level, the greater the priority): an event is eventually kept only if there is no simultaneous event with a lower level than it. In particular, events involving a change to the `status` or `resolution` hide `assignee`/`priority`/`severity` updates, which, in their turn, hide changes to the `milestone` or `CC`.

A *merging* rule is defined in Table 1 only for 1-level simultaneous events, which states that, whenever the `status` and `resolution` of a bug are modified contemporarily, the respective events must be merged into a single macro-event,

which is labelled with the concatenation of their associated activity labels. For example, this implies that the ninth and tenth events in Fig. 1 will be merged together, and labelled with the string "`state`:=*resolved* + `resolution`:=*fixed*".

Also the default sort rule (shown in the table as an ordering relation $<$) only depends on the *what* field, and states that events involving attribute `milestone` must precede those concerning `CC`, and that `priority` (resp., `severity`) updates must precede `severity` (resp., `assignee`) ones. In this way, e.g., the first two events in Fig. 1 will be switched with one another.

Table 1. Default macro-event criterion: predominance, merging and sort rules over simultaneous events. No merging rules for levels 2 and 3, and no sort rules for level-1 events are defined.

Predominance levels		Merging rules	Sort rules
(lev.)	(bug attributes)	(macro-event activity label)	(ordering relation)
1	`status`, `resolution`	$\alpha(\langle _, _, \texttt{status}, _, _\rangle)$+ "+" + $\alpha(\langle _, _, \texttt{resolution}, _, _\rangle)$	—
2	`priority`, `severity`, `assignee`	—	`priority`<`severity`<`assignee`
3	`milestone`, `CC`	—	`milestone`<`CC`

Example 1. Let us apply all default log abstraction operators introduced above (i.e. the event abstraction function in Eq. 1 and the macro-event criterion of Table 1) to the bug trace τ_{ex} encoding the events in Fig. 1. For the sake of conciseness, we only consider events involving the attributes in Table 1. The resulting trace τ'_{ex} consists of 8 events, which are associated with the following activity labels, respectively: l_1="Δ`milestone`", l_2="Δ`CC`", l_3= "Δ`assignee`", l_4= "Δ`CC`", l_5= "Δ`CC`", l_6= "Δ`assignee`", l_7= "`status`:=*resolved*+`resolution`:=*fixed*", l_8= "`status` := *closed*", where l_i denotes the activity label of $\tau'_{ex}[i]$, for $i \in \{1, \ldots, 8\}$. The respective timestamps (at 1-hour granularity) of these events are: $t_1 = t_2$ =(*2012-06-20 10EDT*), t_3 =(*2012-06-20 22EDT*), t_4 =(*2012-06-21 11EDT*), t_5 =(*2012-06-25 05EDT*), t_6 =(*2012-07-02 10EDT*), t_7 =(*2012-07-02 15EDT*), t_8 =(*2013-01-03 11EST*). ◁

5 Trace Manipulation Operators

By applying the event manipulation operators introduced in the previous section, each bug history can be turned into a "process trace", i.e. a sequence of resolution tasks. In order to profitably apply process-oriented prediction techniques, such a trace must be converted into some abstract state-oriented form, capturing somewhat the stage reached by the corresponding bug case along the resolution process. On the other hand, the accuracy of fix-time prediction can be empowered by taking advantage of context information stored for each case, in the form of native or derived trace attributes. These two issues are discussed in details next, in two separate subsections.

5.1 State-Oriented Trace Abstraction

For each trace τ, a collection of relevant prefixes (i.e. sub-traces) $rp(\tau)$ is selected, in order to extract an abstract representation for the states traversed by the associated bug, during its life. Two strategies can be adopted to this end, named *event-oriented* and *block-oriented*. In the former strategy all possible τ's prefixes are considered, i.e. $rp(\tau) = \{\tau(i) \mid i = 1 \dots len(\tau)\}$, whereas in the latter only prefixes ending with the last event of a "macro-activity" are selected, i.e. $rp(\tau) = \{\ \tau(i) \mid 1 \leq i \leq len(\tau)$ and $when(\tau[j]) > when(\tau[i])\ \forall j \in \{i+1, \dots, len(\tau)\}\ \}$.

Independently of the selection strategy, each trace τ' in $rp(\tau)$ is turned into a tuple $state^{\alpha}(\tau')$, whose attributes are all the abstract activities produced by a given event abstraction function α (e.g., that in Eq. 1). The value taken by each of these activities, say a, is denoted by $state(\tau')^{\alpha}[a]$ and computed as follows:

$$state^{\alpha}(\tau')[a] = \texttt{SUM}(\{\ \delta(\tau'[i]) \mid \alpha(\tau'[i]) = a \text{ for } i = 1..len(\tau')\}) \tag{2}$$

where δ is a function assigning an integer weight to each event, based on its properties; by default it is *(i)* $\delta(e) = |added(e)|$, if e is not an aggregation of multiple simultaneous events (i.e. it corresponds to one raw update record) and e involves a multivalued attribute (like `CC, seeAlso`), or *(ii)* $\delta(e) = 1$ otherwise.

Any prefix trace τ' is hence encoded by an integer vector in the space of the abstract activities extracted by α, where each component accounts for all the occurrences, in τ', of the corresponding activity. Such a vector captures the state of a bug (at any step of its evolution) through a summarized view of its history.

Example 2. Let us consider the trace τ'_{ex} shown in Example 1. The unfolding of this trace gives rise to 8 distinct prefix sub-traces, denoted by $\tau'_{ex}(1]$, $\tau'_{ex}(2]$, …, $\tau'_{ex}(8]$. Five abstract activities occur in these traces:

a_1="Δ`milestone`",

a_2="Δ`CC`",

a_3="Δ`assignee`",

a_4="`status`:=*resolved*+`resolution`:=*fixed*",

a_5="`status` :=*closed*".

As concerns trace abstractions, all components of $state^{\overline{\alpha}}(\tau'_{ex}(1])$ (i.e. the tuple encoding the state reached after the first step) are 0 but that associated with a_1 and a_2, which are $state^{\overline{\alpha}}(\tau'_{ex}(1])[a_1] = 1$, and $state^{\overline{\alpha}}(\tau'_{ex}(1])[a_2] = 2$. If using the event-oriented strategy, the above traces will generate 8 state tuples:

$state^{\overline{\alpha}}(\tau'_{ex}(1])=\langle 1,2,0,0,0\rangle$, $state^{\overline{\alpha}}(\tau'_{ex}(2])=\langle 1,2,1,0,0\rangle$,
$state^{\overline{\alpha}}(\tau'_{ex}(3])=\langle 1,4,1,0,0\rangle$, $state^{\overline{\alpha}}(\tau'_{ex}(4])=\langle 1,5,1,0,0\rangle$,
$state^{\overline{\alpha}}(\tau'_{ex}(5])=\langle 1,5,2,0,0\rangle$, $state^{\overline{\alpha}}(\tau'_{ex}(6])=\langle 1,5,2,1,0\rangle$,
$state^{\overline{\alpha}}(\tau'_{ex}(7])=\langle 1,5,2,1,1\rangle$. ◁

Such a state-oriented representation of a log L will be eventually exploited to induce a fix-time prediction model (i.e. a $FTPM$) for L, as explained in the next section.

5.2 Insertion of Additional Trace Attributes

In order to possibly improve the accuracy of discovered prediction models, we want to give the analyst the possibility to inject additional attributes, accounting for the context where resolution tasks took place.

More specifically, our framework is meant to supports different kinds of operators for inserting additional data into bug traces, or for transforming a given (raw or derived) bugs/events attribute, and making it more expressive: *(i) indicators' derivation* operators, *(ii) attribute enrichment* operators, and *(iii) attribute summarization* operators.

Indicators' Derivation. The first kind of operators is essentially meant to compute some statistics over some suitable trace collection. In fact, the insertion of such additional data was already considered in previous bug analysis works [8,9]. We defined five standard indicator derivation operators, which allow for automatically inserting the following fields into each given trace τ, respectively: *(i)* a workload measure indicating the overall number of cases currently opened in the system globally, and several variants of it, computed over specific groups of cases (e.g., all bugs regarding the same product version, component or OS as τ); *(ii)* an analogous collection of counters for the cases fixed in the past year (globally, and for the version/component/OS associated with τ); *(iii)* a "reputation" coefficient, computed for the reporter of τ as in [2]; *(iv)* the average fix time for various groups of related cases (e.g., all bugs concerning the same project or reporter as τ) and closed in the past year; *(v)* several seasonality dimensions (such as, week-day, month, and year) derived from the date of the last event in τ.

Attribute Enrichment. This class of operators is meant to extend the values of an attribute with correlated information, extracted from the same repository. As a standard attribute enrichment mechanism, we consider the possibility of replacing users' identifiers (possibly appearing, e.g., in *who* fields of bug history records, or in certain bug attributes) with their respective e-mail addresses — actually, no further information on people is available in many real bug repositories. To implement such an operator, a greedy matching procedure was developed, which founds on comparing any user ID with all the email addresses appearing in various attributes of the bugs.

Clustering-based Attribute Summarization. These operators aim at reducing the dimensionality of an attribute by partitioning its domain into classes. In particular, in order to get an effective level of abstraction over a trace attribute, say F, we considered the problem of finding a partition of F's domain exhibiting high correlation with fix-time measurements, based on a given aggregation hierarchy H over F. Let $dom(F)$ be the domain of F, and $h \in \mathbb{N}$ be the height (i.e. the number of levels) of H. Let also $H(i)$ be the *i-th* level of H (for $i = 0, l \ldots, h$), and $values(H, i)$ be the values forming that aggregation level. Each level of H corresponds to a partition of $dom(F)$, with a distinguished aggregation degree;

in particular, level 0 (H's leaves) just encodes the raw values in $dom(F)$ without any aggregation, and level h just consists of one node (H's root). Such a hierarchy may be already available for the attribute (this is the case of email addresses and software products, which follow an implicit meronomical and taxonomical scheme, respectively), or it can be computed automatically through clustering methods. Let C_H be a cut of the given hierarchy H, and $values(C_H)$ be the associated nodes, i.e. the set of aggregated F's values identified by C_H. Any such a cut can be associated with a loss measure, denoted by $Loss(C_H)$, w.r.t. the target metrics μ_F, as follows:

$$Loss(C_H) = \frac{\sum_{\tau \in \mathcal{P}(L)} \left[\hat{\mu}_F(\tau[len(\tau)]) - \tilde{\mu}_F(\tau, C_H)\right]^2}{|\mathcal{P}(L)|}$$

where $\tilde{\mu}_F(\tau, C_H)$ is average performance value over all trace prefixes owning a value of F that was assigned to the same C_H's node as τ. An optimal cut of H is one minimizing this loss measure, among all those satisfying a user-defined property. A greedy procedure has been implemented in our framework for computing such a cut, based on a maximal constraint on cut's size (i.e. number of nodes in the cut).

Text-oriented Attribute Summarization. In many real application scenarios, bug/issue handling systems store free-text contents for each case, ranging from notes to messages. In particular, in the case of Bugzilla's bug repositories, a *summary* field is kept for each bug case, providing a short description of the problem occurred. Let us assume that the textual contents of any bug τ are stored into a vector $text(\tau)$, encoding a bag-of-word representation for the concatenation of all τ's free-text fields, based on classic vector-space model and TF-IDF weights. Clearly, such a textual description can hold important information on the nature and characteristics of a bug, and it can help improve the estimation of its fix time, as pointed out in [6]. However, as the vocabulary used in such descriptions may be rather large and sparse, extending the attributes of each trace with a mere bag-of-word view of its text contents will hardly improve the accuracy of fix-time predictions; indeed, such an overly detailed representation might even confuse the learner and lead to overfitting and imprecise models.

In order to face this issue, we introduced a further kind of text-oriented attribute-summarization operators, meant to grasp a concise high-level representation of text contents. The standard implementation of this operator class relies on computing a low-rank approximation of the term-by-bug matrix, based on Singular Value Decomposition (SVD) techniques [12]. The mapping of cases to the underlying concept space (i.e., the U matrix in the $U \times \Sigma \times V^T$ factorization of the term-by-bug matrix) is used to build a low-dimensional (and more abstract) representation of text contents. Such summarized "latent-semantics" vectors will be used to complement the attributes of each trace, so obtaining a richer description of the corresponding bug/issue case.

6 *FTPM* Discovery Algorithm

This section is meant to offer a summarized view of our whole approach to the discovery of a Fix-time Prediction Model ($FTPM$), based on a given set of raw bug records.

The approach is illustrated in Fig. 2 as a meta-algorithm, named `FTPM Discovery`, encoding the main steps of our (process-oriented) data-transformation methodology, and the eventual application of a predictive induction method to the transformed log.

The algorithm takes as input a bug/issue tracking repository, storing a collection of records (like those at the beginning of Sect. 3), and some parameters regarding the application of data-manipulation operators. In order to apply the abstraction operators introduced in Sect. 4, these data are first turned into a set of traces (i.e. a log).

Based on a given *filtering* criterion Φ, function `filterEvents` possibly removes uninteresting events (e.g., outliers or noisy data), which may confuse the learner.

Function `handleMacroEvents` allows us to apply a given *macro-event* criterion Γ (like that described in Table 1) to rearrange each group of simultaneous log events according to the associated *predominance, reordering* and/or *merging* rules.

Steps 4 and 5 are meant to possibly associate each trace τ with additional "derived" data, to complement the original contents of $data(\tau)$ with context information.

Input: Collection B of bug/issue records, filtering criterion Φ, a macro-event criterion Γ, event abstraction function α, and prefix selection strategy $S \in \{$`BLOCK, EVENT`$\}$
Output: An $FTPM$ (Fix-time Prediction Model) for B
Method: Perform the following steps:
1 Convert B into a log L of traces;
2 $L :=$ `filterEvents`(L, ϕ);
3 $L :=$ `handleMacroEvents`(L, Γ);
4 $L :=$ `deriveTraceAttributes`(L);
5 $L :=$ `refineTraceAttributes`(L);
6 **if** $S =$ `BLOCK` **then**
7 RS:=$\{\tau(i) \mid \tau \in L, 1 \leq i \leq len(\tau)$, and $when(\tau[j]) > when(\tau[i])\ \forall j \in \mathbb{N}$ s.t. $i < j \leq len(\tau)\}$;
8 **else**
9 RS:= $\{\tau(i) \mid \tau \in L$, and $1 \leq i \leq len(\tau)\}$;
10 **end if**
11 $M :=$ `mineFTPM`(RS, α);
12 **return** M.

Fig. 2. Meta-algorithm `FTPM Discovery`.

Basically, function `deriveTraceAttributes` is devoted to insert new trace attributes, with the help of *indicators' derivation* operators and of *attribute enrichment* operators like those defined in the previous section.

Conversely, function `refineTraceAttributes` is meant to transform a number of (raw or derived) cases'/events' attributes, and eventually them into more expressive ones, by way of *attribute-summarization* operators.

Steps 6–10 are simply meant to extract a set RS of relevant (sub-)traces out of $\mathcal{P}(L)$, based on the chosen event selection strategy S. RS is then used by function `mineFTPM` as a training set, in order to eventually induce an $FTPM$. To this end, as explained in Sect. 4, each trace $\tau \in RS$ is converted into a tuple labelled with the fix-time measurement $\mu_F(\tau)$, and encoding both the representation of τ's state (w.r.t. the given event abstraction function α), and its associated (augmented) data tuple $data(\tau)$. More precisely, $data(\tau)$ and $state^{\alpha}(\tau)$ are used as descriptive/input attributes, while regarding the actual remaining-time value $\mu_F(\tau')$ as the target of prediction.

At this point, various learning methods (including those mentioned in Sect. 2) can be used to induce a regression or classification model. Different solutions for carrying out this task have been implemented in our prototype system, as described next.

7 The Implemented System

A prototype system was developed to fully implement the approach above, and support the discovery and analysis of $FTPM$s. The system allows the analyst to flexibly and modularly apply all the data-processing operators presented in this work, in an interactive and iterative way, as well as to define and store (in a reusable form) new variants of them, according to a template-based paradigm.

The conceptual architecture of the system is shown in Fig. 3. System's functionalities are organized in three layers, which concern respectively: the storage, handling and manipulation of log data (*Data Management and Processing*), the implementation of core data mining techniques (*Data Mining*), and the handling and evaluation the prediction and clustering models found with these techniques (*Models' Management*).

The *Log Repository Gateway* module supports the importation of historical data stored in a remote bug/issue tracking repository. Its current implementation allows for connecting to the web interface of any Bugzilla repository, and converting the histories of selected bugs into bug traces. The imported log, possibly cleaned according to suitable filtering rules, is stored in the *Event Log* archive.

The *Event Manipulation* and *Trace Manipulation* modules provide various data-transformation mechanisms (possibly already stored in the form of *Event/Trace Transformation Rules*), in order to produce enhanced views of the log, which are more suitable for fix time prediction. More specifically, the first module is responsible for manipulating groups of simultaneous events according to a desired macro-event criterion (sub-module *Macro-events' manipulation*), as discussed in Sect. 4, and for extracting high-level activity labels from event prop-

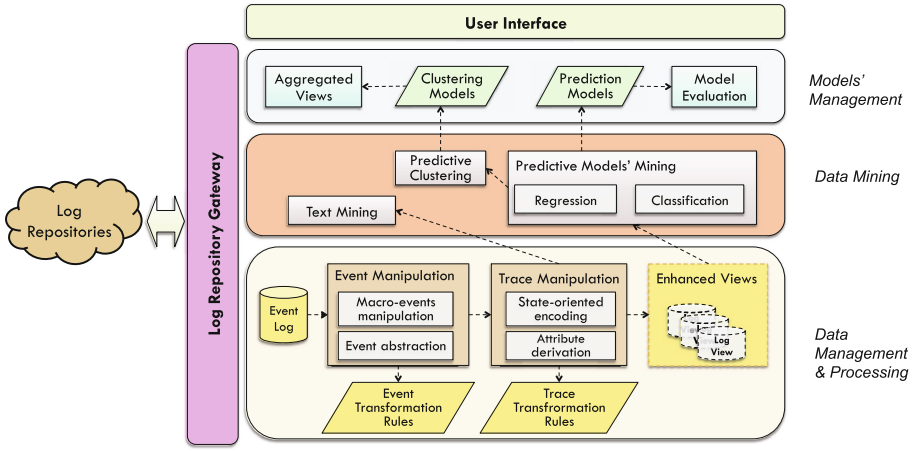


Fig. 3. Logical architecture of the implemented system.

erties (sub-module *Event abstraction*). On the other hand, the *Trace Manipulation* module allows to build a state-oriented abstraction for each selected trace (sub-module *State-oriented encoding*), while possibly enriching each trace with additional (derived and/or summarized) attributes (sub-module *Attribute derivation*), as explained in Sect. 5.2. In particular, one can extract high-level trace attributes capturing the semantics of text case fields, based on an SVD analysis of the term-by-case matrix. This task leverages the *Text Mining* module (from the *Data Mining* layer), implementing core text processing functionalities (including tokenization, stemming, TF-IDF encoding), and SVD calculations.

Each "refined" log view, obtained with of the above described functionalities. can be used as input for module *Predictive Models' Mining*. To this end, the abstracted traces are delivered to either the *Regression* or *Classification* submodule, depending on which learning task was chosen by the user. Preliminary to the induction of a prediction model, the traces can be possibly partitioned into groups by one of the predictive clustering methods implemented by the *Predictive Clustering* module, which will also produce a set of decision rules for discriminating among the discovered clusters. In this case, each cluster will be eventually equipped with a distinct fix-time predictor.

All clustering and predictive models discovered can be explored and stored for subsequent analyses. The *Models' Management* layer also supports the evaluation of prediction models via several accuracy metrics (e.g., *rmse*, *mae*, *precision*, *recall*). Moreover, the *Aggregated views* module can compute aggregate statistics and graphics, concerning the distribution of any trace attribute in any trace cluster (like that in Fig. 4).

Regression Algorithms. Several alternative learning methods are currently implemented in our system, which support the induction of a $FTPM$, from a propositional training set like that described in the previous section. These methods,

ranging from classical regression methods to state-aware Process Mining ones are listed next:

- `IBK`, a lazy (case-based) naïve regression method available in Weka [12], used with $k = 1$, Euclidean distance, and nominal attributes's binarization;
- `RepTree`, implementing the homonymous regression-tree method [12], while using variance reduction and 4-fold reduced-error pruning;
- `CATP`, implementing the approach in [4], which first partitions the traces into clusters, via a multi-target predictive clustering strategy, and then equips each cluster with an "A-FSM" predictor, as in [13] (using no history horizon and the bag-oriented trace abstraction);
- `AATP-IBK` and `AATP-RepTree` [5], which still adopt a multi-target predictive clustering scheme for partitioning fully-unfolded traces, and regressors `IBK` and `RepTree`, respectively, as the base learner for each discovered cluster.

Classification Algorithms. Like in previous bug analysis works, the analyst can also induce a classification model for the prediction of (discrete) fix times, after defining a set of a-priori classes in terms of fix-time ranges (possibly with the help of automated binning tools). To this end, a number of existing classifier-induction algorithms can be exploited, including popular decision-tree learning algorithm *J*48 (with 3-fold reduced error pruning), and the `Random Forest` Algorithm [3] (with size 10).

8 Experimental Evaluation

We performed a series of tests with our prototype system against two real datasets: a log from project *Eclipse*'s Bugzilla repository, and a collection of traces from a problem management system *BPI Challenge* [11]. In all tests, `FTPM Discovery` was used to find different numeric fix-time predictors, by varying the underlying regression algorithm, combined with the event-based prefix selection strategy (i.e. $S =$ `EVENT`).

The accuracy of the discovered models was evaluated with the standard error metrics *root mean squared error (rmse)* and *mean absolute error (mae)* — both computed via 10-fold cross validation, and normalized by the average fix time (i.e. 59 and 179 days for bug cases and problem management cases, respectively), for ease of interpretation.

8.1 Experiments on Bug Logs of Project Eclipse

A sample of 3906 bug records (gathered from January 2012 to March 2013) was turned into a set of bug trace like those described in Sect. 3. The length of full bug traces ranged from 2 to 27, while bug fix time ranged from one day (i.e., a bug is opened and closed in the same day) to 420 days, with an average of about 59 days.

Log Manipulation Steps and Enhanced Log Views. In order to put this log into a more suitable for prediction, we applied some of the log manipulation mechanisms defined in our framework. Five views of the original log, named $L_0, \ldots, L_4$, were produced by incrementally applying the basic event abstraction function $\overline{\alpha}$ in Eq. 1 and all the data-processing functions appearing in algorithm `FTPM Discovery` (cf. Figure 2).

- The basic "abstracted" view L_0 was obtained by only applying the event abstraction function $\overline{\alpha}$ to the selected bug traces;
- A "cleaned" view L_1, consisting of 2283 traces, was produced making L_0 undergo a specific instantiation of function `filterEvents`, removing the following data: *(i)* bugs never fixed, *(ii)* "trivial" bug cases (i.e. all bugs opened and closed in the same day), and *(iii)* trace attributes (e.g., `version`, `whiteboard`, and `milestone`) featuring many missing values, and bug/event fields (e.g. `summary`) containing long texts.
- View L_2 was produced by restructuring the simultaneous events in L_1 with the default implementation of function `hanldeMacroEvents` (based on the rules of Table 1).
- View L_3 was obtained by applying all the derivation mechanisms described in Sect. 5. As a result, each trace is equipped with the new attributes: `week-day`, `month`, and `year`; indicators `workload`, `workload_version`, `workload_component`, `workload_OS` (i.e. the number of bugs open in the system in general, an for the same product version, component and operating system, resp.); `reporter_avgFT`, `project_avgFT` (average fix-time for all past years' bugs with the same reporter or project, resp.); coefficient `reporter_reputation` [2].
- View L_4 was obtained by the built-in implementation of `abstractTrace-Attributes`, applied to L_3. In particular, all people identifiers in the `reporter` bug attribute were replaced with a number of reporters' groups representing different organizational units (namely, {*oracle, ibm.us, ibm.no_us, vmware, other*}), and extracted semi-automatically from e-mail addresses. A similar approach was used to produce a binary abstraction (namely {*eclipse, not_eclipse*}) of attribute `assignee`, and an aggregate representation of both `product` and `component`. Each bug was finally equipped with a series of attributes encoding a low-dimensional representation of its textual contents, computed with the help of the SVD-based technique described before.

Prediction Results. Table 2 reports the normalized *rmse* and *mae* errors obtained, with different regression methods available in our system, on the five log views described above. Two different learning setting were considered to this end: using bug history information (originally registered in terms of attribute-update events), and neglecting it. Note that the latter setting is intended to provide the reader with a sort of baseline, mimicking the approach followed by previous fix-time prediction works.

In general, it is easily seen that results obtained in the "no bug history" setting — where only the initial data of reported bugs are used as input variables

Table 2. Regression results on Eclipse bugs. Rows correspond to different *FTPM* induction methods, tested in two learning settings: without and with bug history events. Columns $L_0, \ldots, L_4$ correspond to different views of original data, obtained with our log manipulation operators.

Predictors		rmse					mae				
Setting	*Methods*	L_0	L_1	L_2	L_3	L_4	L_0	L_1	L_2	L_3	L_4
No Bug History (Baseline)	IBK	1.051	1.051	1.050	1.092	1.093	0.569	0.569	0.561	0.583	0.584
	RepTree	0.973	0.973	0.970	0.966	0.925	0.562	0.562	0.552	0.547	0.546
	Avg (no history)	**0.973**	**0.973**	**0.970**	**0.966**	**0.925**	**0.562**	**0.562**	**0.552**	**0.547**	**0.546**
History-aware	CATP	0.967	0.873	0.880	0.737	0.640	0.510	0.467	0.440	0.380	0.316
	IBK	0.983	0.823	0.807	0.793	0.803	0.430	0.360	0.360	0.347	0.345
	AATP-IBK	1.003	1.007	0.827	0.800	0.710	0.437	0.473	0.367	0.353	0.305
	RepTree	1.013	0.883	0.907	0.910	0.773	0.533	0.473	0.473	0.477	0.351
	AATP-RepTree	0.970	0.930	0.887	0.783	0.657	0.510	0.530	0.437	0.390	0.306
	Avg (history-aware)	**0.987**	**0.903**	**0.862**	**0.8059**	**0.717**	**0.484**	**0.461**	**0.415**	**0.389**	**0.325**

for the prediction — are rather poor, if compared to the average ones obtained in the "history-aware" setting. Indeed, the errors measured in the former setting are rather high, no matter which inductive method (i.e. IBK or RepTree) is used, and which pre-processing operations are applied to the original logs. Interestingly, this result substantiates our claim that exploiting bug activity information helps improve the precision of fix-time forecasts.

On the other hand, in the second setting, both *rmse* and *mae* errors tend to decrease when using more refined log views. In particular, substantial reductions were obtained with the progressive introduction of macro-event manipulations (view L_2), and of derived and abstracted data (views L_3 and L_4, respectively).

Very good results are obtained (in the history-aware setting) when using some kind of predictive clustering method, be it single-target (RepTree) or multi-target (CATP, AATP-RepTree and AATP-IBK). However, trace-centered clustering approaches (namely, CATP, AATP-RepTree, AATP-IBK achieve better results than RepTree, which considers all possible trace prefixes for the clustering. The benefit of using a clustering procedure is quite evident in the case of IBK, which generally gets worse achievements than any other approach, presumably due to its inability to fully exploit derived data. Indeed, still focusing on the history-aware setting, we note that IBK gets more precise when embedded in the predictive clustering scheme of AATP-IBK.

8.2 Experiments on the Problem Management Log

The analyzed log comes from a problem handling system (named *VINST*), offered by Volvo IT Belgium as a benchmark dataset [11] for 2013 BPI Challenge.

The main attributes of each problem resolution case p are: p's impact (i.e., medium, low, high); the product affected by p; the support team currently solving p, and the associated functional division and organization line;

Table 3. Regression results on the VINST problem management system's logs. Rows and columns still correspond to different *FTPM* induction methods and different log views, respectively.

Predictors		rmse		mae	
Setting	*Methods*	L'_0	L'_1	L'_0	L'_1
No Bug History (Baseline)	`IBK`	0.924	0.818	0.499	0.363
	`RepTree`	0.937	0.539	0.523	0.245
	Avg (no history)	**0.931**	**0.679**	**0.511**	**0.304**
History-aware	`CATP`	0.930	0.563	0.542	0.278
	`IBK`	0.916	0.802	0.466	0.357
	`AATP-IBK`	0.914	0.613	0.465	0.242
	`RepTree`	0.888	0.508	0.470	0.235
	`AATP-RepTree`	0.887	0.467	0.466	0.205
	Avg (history-aware)	**0.907**	**0.591**	**0.482**	**0.265**

the country where the support team is located (`organization country`); the `resource` currently operating on p and its nationality (`resource country`); the `status` and `substatus` of p. More specifically, the `status` of a problem may take the following values: *accepted*, *queued*, *completed*, and *closed*. Attribute `substatus` gives finer grain information on the resolution stage reached by a problem, and can be: *assigned*, *awaiting_assignment*, *cancelled*, *in_progress*, *wait* (i.e. information from third parts is needed to correctly diagnose the issue), *unmatched*, and *closed*. With respect to the formalization of Sect. 3, for any event e, *what*(e) takes the form of a pair $\langle$`status`, `substatus`$\rangle$ (i.e. both case attributes are modified contemporaneously), while *added*(e) (resp., *removed*(e)) stores the new (resp., old) values associated with these attributes.

Log Manipulation Steps and Enhanced Log Views. A sample of 1487 issue records (from January 2006 to May 2012) was turned into a set of traces, with lengths ranging from 3 to 37, and fix times ranging from 1 to 2255 days (179 days in the average).

By applying only a subset of our log manipulation operators, we generated two (slightly) refined views of the original log, named L'_0 and L'_1, as follows:

- View L'_0 was obtained by only applying, to the selected problem traces, a basic event abstraction function, just amounting to extracting a combination of the values assigned to attributes `status` and `substatus`;
- View L'_1 was produced by equipping each trace with several attributes derived from its starting time: the global `workload` indicator (i.e. the number of problems open at that time), and the starting `week-day`, `month` and `year`. Each trace was also enriched with an attribute `firstOrg`, indicating which organization line was initially chosen to work on it. Neither filtering nor macro-events restructuring was employed to generate this view.

Prediction Results. Table 3 shows the normalized *rmse* and *mae* errors obtained, with different regression methods, on the two log views described above, still considering two learning settings: with and without information on the past history of a case.

Similarly to the bug tracking scenario, a neat decrease in both error measures is achieved when working with the more refined log view L'_1 (w.r.t. L'_0), even though this latter was obtained by just using very simple log manipulation operators. Moreover, the performances of baseline predictors seem to get improved when exploiting information on past case history ("history-aware" setting) and/or some kind of predictive clustering method (compare `AATP-RepTree` and `AATP-IBK` to `RepTree` and `IBK`, respectively).

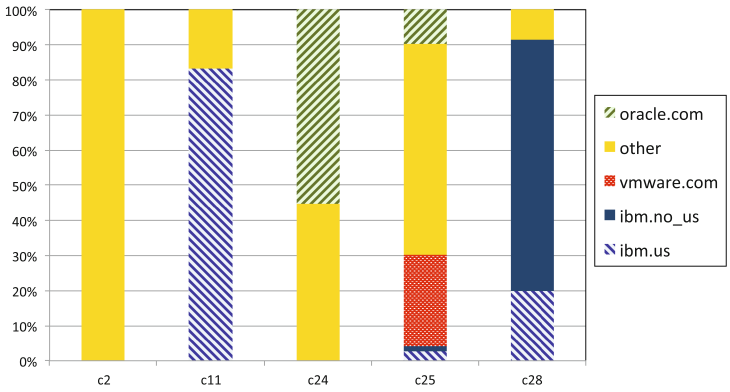


Fig. 4. Aggregated statistics for some trace clusters found (with `AATP-IBK`) on the Eclipse log.

Table 4. Clusters found on the bug scenario (left) and the problem management scenario (right).

id	condition
c_2	`Reporter_avgFT` $>$ 137.4 days $\wedge$ `workload` $>$ 600 $\wedge$ `Component` $\in$ {*Data*, *LuaTools*, *Graphiti*, *MQTT*, *documentation*, *Editor*, *General*, *Define*, *Photran*, *Scout*, *Report*, *runtime*, *RDT*, *Website*}
c_{11}	54.46 days $<$ `Reporter_avgFT` $\leq$ 117.9 days $\wedge$ `Component` $\in$ {*IDE*, *Deployment*, *EDT*, *spdy*, *JavaGen*, *documentation*, *runtime*}
c_{25}	16.57 days$<$ `Reporter_measure` $\leq$ 54.47 days $\wedge$ `Component` $\in$ {*Search*, *Extra*, *Model*, ...} $\wedge$ `Product` $\in$ {*Data*, *JDT*, *EclipseLink*, *STEM*, *Jubula*, *Platform*, *Linux*, *Orbit*, *WTP*, *OSEE*, *MDT*, *PTP*, *Virgo*, *JSDT*, *Orion*}

id	condition
c_1	`product` $\in$ {*PROD659*, *PROD476*, *PROD289*, ...} $\wedge$ `year` $>$ 2011 $\wedge$ `workload` $>$ 437
c_6	`product` $\in$ {*PROD378*, *PROD660*, *PROD473*, ...} $\wedge$ `year` $>$ 2011
c_8	`product` $\in$ {*PROD276*, *PROD277*, *PROD372*, ...} $\wedge$ `firstOrg` $\in$ {*Line C*, *Line B*, *Line V2*, *Line G4*} $\wedge$ `year` $>$ 2011 $\wedge$ `workload` $>$ 219
c_{12}	`product` $\in$ {*PROD659*, *PROD793*, *PROD597*, *PROD96*, *PROD671*, *PROD231*, *PROD637*, *PROD209*, *PROD13*, *PROD812*, *PROD813*, *PROD153*} $\wedge$ 2010 $<$ `year` $\leq$ 2011 $\wedge$ `workload` $>$ 587
c_{28}	`year` $\leq$ 2010 $\wedge$ `workload` $\leq$ 11

8.3 Qualitative Results: Discovered Clusters and Aggregate Statistics

Table 4 shows the classification rules associated with some of the (about 30) clusters discovered, on each of the application scenarios, when using our approach with `AATP-IBK` as base regressor. Beside supporting accurate cluster-based predictions, these rules offer a readable and interesting description of different context-dependent execution variants discovered for both issue resolution processes. In particular, in the case of Ecplise's bugs, most of the discovered variants mainly depend on the reporter-based average fix time, on the component/product affected, and on the workload indicator. Conversely, the variants discovered for the problem management scenario are mainly linked to the product, the workload and the year when the problem was reported.

Figure 4 shows a chart for the distribution of certain reporters' groups (computed over attribute `reported` with one of ours attribute summarization procedures) in some of the clusters discovered (namely, c_2, c_{11}, c_{24}, c_{25}, c_{28}).

9 Conclusions

We proposed a framework and an associated system for predicting issue/bug resolution times, which fully exploit case attributes' change logs. A rich collection of flexible data-transformation methods allows the analyst to produce high-quality process-oriented logs, and eventually reuse regression/classification to discover a state-based prediction model. Encouraging results obtained on real logs empirically proved the validity of our approach, and the interestingness of discovered models.

As to future work, we plan to extend our approach as to deal with a wider variety of target measures, ranging from Quality of Services measures to process-centric Key Performance Indicators. We will also investigate the application of our methods to the logs of other kinds of data-centric and lowly-structured collaboration environments.

References

1. Anbalagan, P., Vouk, M.: On predicting the time taken to correct bug reports in open source projects. In Proceedings of International Conference on Software Maintenance (ICSM 2009), pp. 523–526 (2009)
2. Bhattacharya, P., Neamtiu, I.: Bug-fix time prediction models: can we do better? In: Proceedings of 8th International Conference on Mining Software Repositories (MSR 2011), pp. 207–210 (2011)
3. Breiman, L.: Random forests. Mach. Learn. **45**(1), 5–32 (2001)
4. Folino, F., Guarascio, M., Pontieri, L.: Discovering context-aware models for predicting business process performances. In: Proceedings of 20th Intlernational Conference on Cooperative Information Systems (CoopIS 2012), pp. 287–304 (2012)

5. Folino, F., Guarascio, M., Pontieri, L.: A data-adaptive trace abstraction approach to the prediction of business process performances. In: Proceedings of 15th International Conference on Enterprise Information Systems (ICEIS 2013), pp. 56–65 (2013)
6. Folino, F., Guarascio, M., Pontieri, L.: Discovering high-level performance models for ticket resolution processes. In: Proceedings of 21st International Conference on Cooperative Information Systems (CoopIS 2013), pp. 275–282 (2013)
7. Giger, E., Pinzger, M., Gall, H.: Predicting the fix time of bugs. In: Proceedings of 2nd International Workshop on Recommendation Systems for Software Engineering (RSSE 2010), pp. 52–56 (2010)
8. Hooimeijer, P., Weimer, W.: Modeling bug report quality. In: Proceedings of 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2007), pp. 34–43 (2007)
9. Marks, L., Zou, Y., Hassan, A.E.: Studying the fix-time for bugs in large open source projects. In: Proceedings of 7th Intlernational Conference on Predictive Models in Software Engineering (Promise 2011), pp. 11:1–11:8 (2011)
10. Panjer, L.: Predicting eclipse bug lifetimes. In: Proceedings of 4th International Workshop on Mining Software Repositories (MSR 2007), p. 29 (2007)
11. Steeman, W.B.P.I.: Challenge : closed problems. Ghent University (2013). doi:10.4121/uuid:c2c3b154-ab26-4b31-a0e8-8f2350ddac11
12. Tan, P.-N., Steinbach, M., Kumar, V.: Introduction to Data Mining. Addison-Wesley Longman, Boston (2005)
13. van der Aalst, W.M.P., Schonenberg, M.H., Song, M.: Time prediction based on process mining. Inf. Syst. **36**(2), 450–475 (2011)